

StockPilot — Warehousing Overview

A briefing for warehouse operations leaders evaluating the system as an operating platform for distribution work.

Prepared for: A partner warehousing expert **Last updated:** 2026-05-20

Executive summary

StockPilot is a purpose-built warehouse + inventory operating system that replaces the ad-hoc Excel sheets, paper pick lists, and disconnected POS-side procurement tools that most small-to-mid-size warehouses run on. It handles the full operational lifecycle of physical inventory:

receive → slot → reserve → pick → pack → stage → ship → sign → reconcile

...and it does it with three things most homegrown systems lack: real-time visibility across multiple warehouses, a tamper-evident audit trail of every quantity change, and a role-based permission model that distinguishes between a viewer, a picker, a manager, and an admin without requiring a separate user-management product.

It runs in the cloud. There is no server room, no nightly backup batch, no on-call DBA. New warehouses can be turned on in minutes, not weeks. Staff use it on a phone or a laptop with no specialized hardware beyond a Bluetooth or built-in barcode scanner.

This is not a vague claim — every capability described below is built and in production at L4L Fresno today.

What it solves, in operations terms

Every warehouse manager has lived with these problems. The map below shows what StockPilot already solves and what it's actively building toward.

Pain point	What hurts	How StockPilot handles it
"Where is the canopy?"	Asset hunting eats hours per week.	A Rentals ledger tracks every checkout: who has it, when it's due back, what condition. Overdue items surface in red.
"Did we actually ship that order?"	Phone-tag with the requester to confirm delivery.	Digital signature capture on a phone or tablet at handoff. Signed PDF is auto-generated and emailed to the requester.
"How many size-XL shirts in DC4?"	Excel cells go stale within hours of a busy day.	A single source of truth in the database. Update on the phone, the laptop reflects it within 250 ms.
"Did somebody pull from DC1 when they	Cross-warehouse leaks become end-of-	Every staff user is scoped to assigned warehouses. They literally can't see, never mind

Pain point	What hurts	How StockPilot handles it
should have pulled from DC4?"	month write-offs.	pull from, warehouses outside their assignment.
"Who took that and didn't write it down?"	Loss without accountability.	Every quantity change is a row in an immutable ledger (stock_movements) with user, timestamp, source, reason.
"What's slow-moving?"	Dead stock occupies racks for years.	A Dead Stock report flags every SKU with on-hand quantity but no out-movement in N days.
"What do we need to reorder?"	Reorder math sits in a manager's head.	Reorder Forecast report ranks every item below its reorder point with the deficit + estimated refill cost.
"What's the value of inventory right now?"	Year-end audit becomes an event, not a report.	Inventory Valuation report runs in seconds. PDF export. Categorized + per-warehouse breakdowns.
"What got picked but didn't ship?"	Reservation-vs-fulfillment gap.	Stock-reservation engine holds picks until they're either shipped, signed, or cancelled. Released reservations re-enter available stock instantly.
"Did the count match?"	Cycle counts are a manual scan-and-compare exercise.	Cycle Count module with variance auto-calculation. AI Shelf Scan turns a photo of a textbook shelf into 30+ ISBN-verified line items in seconds.
"Where's that on the floor?"	New employees can't find anything.	Every item carries a rack label (e.g. 38-A). Pick slips print in rack order so the picker walks one path, not a zigzag.
"Whose request is this?"	Order origination unclear.	Public order link (/r/<token>) lets any teacher request items without a system account. Manager approves; warehouse picks. Origination tied to email + IP + timestamp.

A day in the life — on the floor

Receiving

The receiver scans a barcode on the arriving package or types the SKU. The PO line on the screen lights up with what's expected. Quantity received gets typed. The system writes a **stock_movement** of type **receive_po**, increments **quantity_on_hand**, and timestamps it. Photo of the packing slip can be attached. If the receive quantity doesn't match the PO, a shortage flag is recorded — visible to the manager without anyone having to escalate.

Putting away

After receive, the system suggests a put-away rack (configurable per item — primary location). The receiver walks the item to the rack, scans the rack label to confirm, and the system updates `bin_location`. From this moment forward, that item is findable.

Picking

A pick slip is generated as a PDF when a manager approves an order. The slip lists items in rack order — the picker walks one continuous path through the warehouse, not back-and-forth across the floor. Each line on the slip has the item name, SKU, rack location, quantity requested, and a checkbox.

After the physical pick, the picker opens the order on their phone and confirms what was actually picked (sometimes less than requested if the rack was short). The system writes `stock_movement` rows of type `pick`, decrements stock, and reserves the picked quantity until ship.

Packing & staging

Manager hits "Mark Staged for Pickup" or "Mark Staged for Delivery." Two packing slips generate: one for the warehouse (internal — has rack details), one for the customer (clean copy in the box).

Shipping & signature

For deliveries, the manager hits "Assign Delivery" and picks a driver. The driver gets the order on their phone. When delivered, the driver hits "Collect Signature." The signature page opens with a canvas. The receiver signs, optionally adds their name as a different person collecting on behalf of the requester. Submit, and the order is `completed`. A signed-PDF receipt emails to both the original requester and the signer.

End-of-shift

Manager checks the dashboard. Three tiles up top: open orders, items below reorder point, items shipped today. Everything that needs attention is one click away.

A week in the life — for the manager

Monday morning

- Open the dashboard. See: 14 open orders, 6 below reorder point, 3 overdue rentals.
- Open the rentals overdue tab. Call those three employees. Update their rentals as returned once they're back.
- Open the reorder forecast report. Generate a PO for the top 5 short SKUs in one batch.

Mid-week

- New employee starts. Add them to the Team page as `staff`, assign them to DC4 (warehouse scoping). They can immediately log in and pick orders for DC4 but cannot see DC1 inventory.
- A new vendor sends a quote. Add them as a supplier under Suppliers. Existing items can be re-linked to the new supplier without disturbing PO history.

End of month

- Open the Inventory Valuation report. Export to PDF. Send to the business office.
- Open the Stock Movement Summary report. Confirms top movers + variance.
- Spot one item with high variance: open it, scroll through stock_movements. Trace it back to two short-pick events two Fridays ago. Discuss with the picker.

Quarterly

- Schedule a wall-to-wall cycle count. Start a session, scope to warehouse DC4, kick it off Monday morning.
- Staff scan-counts each rack. For book shelves, snap a single photo per shelf — AI Shelf Scan returns ISBN-matched counts in 5-10 seconds per shelf. Manual verify the low-confidence ones.
- Post counts. Variance becomes adjustment entries in **stock_movements**. Reconciled stock + audit trail for accounting.

What's in the box (capabilities)

Multi-warehouse, with scope enforcement

A single workspace can hold any number of warehouses (DC1, DC2, ...). Each item lives in exactly one warehouse. Stock numbers, racks, and reorder math are per-warehouse — never blurred across.

Crucially, **scope is enforced at the database layer**, not just hidden in the UI. A staff user assigned to DC4 sends a request from a browser claiming "show me DC1 inventory" — the database itself returns zero rows. There is no path through the app that would let them see what they shouldn't. This is called Row-Level Security and it's the operational difference between "trust the app" and "trust the database."

Item catalog with photos, racks, sizes, and grades

Every item has:

- Name, SKU, barcode, description
- Quantity on hand, reorder point, reorder quantity, unit cost
- Warehouse, primary location, **rack/row** (and for books, crate color + crate number)
- Item type (product, book, asset, consumable)
- Up to N uploaded photos (compressed client-side to keep storage costs flat)
- Tags + categories + custom fields for org-specific needs

For sized swag (t-shirts), bulk-create generates one row per selected size (XS through XXXXXL) in one click — each variant gets its own SKU like **SP-0KX68-UAA-XL**.

Bundles / kits

Group multiple items into a "bundle" (e.g., new-hire welcome kit = 1 shirt + 1 lanyard + 1 notebook). When you distribute a bundle, the system auto-decrements each component. If a component is short, the run is flagged in the Bundle Shortages report.

Orders — request → approve → pick → ship → sign

The order workflow is the longest-running flow in the system. State machine guarantees you can't ship something that wasn't picked, can't sign something that wasn't shipped. Public link (`/r/<token>`) means external requesters don't need accounts — they just visit a URL, fill the form, get a confirmation email, click the link, the order shows up in the manager's queue.

Every transition is logged with user + timestamp. The full timeline of an order is visible on its detail page.

Rentals — circulating-asset accountability

A separate inventory class for items that go out and come back: canopies, AV gear, event supplies. No money — pure accountability.

- Pick the borrower (team member from the directory OR free-text name for non-system people)
- Pick items + quantities from the rental catalog
- Set the expected return date
- Submit — items are reserved, unavailable for ordering until returned

On the activity list, overdue rentals surface in red ("Overdue by 3 days"). Mark returned with one click; items release back to available stock automatically.

Rental items live in a separate catalog from regular inventory — a canopy never accidentally shows up on an order picker, and a textbook never accidentally shows up on a rental form.

Cycle counts + AI shelf scan

Standard cycle count: manager defines scope (warehouse, categories, locations), staff counts on a phone, system computes variance against system stock, variances become `stock_movement` adjustments.

AI Shelf Scan is a unique capability: take one photo of a textbook shelf with your phone, Google Gemini reads the spines, returns ISBN matches with confidence scores. Matches above 0.85 confidence auto-populate the count lines. Average single-photo turnaround: 4-8 seconds for 25-30 books.

This collapses what used to be a 20-minute manual scan per shelf into 30 seconds of photo + review. Used at L4L Fresno for the textbook room cycle count — confirmed 95%+ accuracy in shipped audits.

Stock movements — the immutable ledger

Every quantity change in the system writes a row to `stock_movements`. Receives, picks, transfers, adjustments, initial loads, bundle distributions, rentals, returns. The row records:

- Item, quantity_change, previous_quantity, new_quantity
- Movement type (receive_po, pick, transfer, adjust, etc.)
- From/to location (for transfers)
- User who did it, timestamp
- Reason / notes
- Reference (the PO, the order, the rental, the cycle count session) so you can trace any movement to its triggering event

We never delete from this table. Adjustment to a row goes via a new row (an offsetting movement). The result is a complete, tamper-evident history that satisfies audit, compliance, and end-of-period

reconciliation in one query.

Purchase orders + import

Create POs from scratch, or generate a PO directly from the Reorder Forecast (one click → draft PO with every item below reorder point, supplier-grouped, suggested quantities). Receive against a PO and the receive flow auto-suggests the right line.

PO Import takes a vendor's emailed PDF/CSV invoice, parses it via a small AI step, and creates the PO draft + lines automatically. Saves 5-15 minutes per invoice that used to be hand-keyed.

Reports + PDFs

Nine reports, every one with CSV + PDF export:

1. **Inventory Valuation** — current \$ value across warehouses + categories
2. **Stock Movement Summary** — by movement type, top movers, last 30 days
3. **Reorder Forecast** — items at/below reorder point + estimated refill cost
4. **Shrinkage & Adjustments** — negative adjustments + cost impact, last 30 days
5. **Supplier Scorecard** — on-time rate, lead time, fill rate, spend, last 90 days
6. **Velocity Classes (ABC)** — items ranked A→D by dollars-out; identifies focus stocking + dead stock
7. **Dead Stock** — on-hand items with no out-movement, ranked by carrying cost
8. **Bundle Activity** — distribution runs by bundle, kits out, component cost out
9. **Bundle Shortages** — components that ran short during distribution

PDFs include item photos inline so the warehouse manager isn't squinting at SKUs trying to remember what **SP-RUQPL-LJC-2** looks like.

Mobile

Native iOS + Android app (Expo / React Native). Same login as the web. Optimized for:

- In-warehouse scanning (barcode-driven receive, pick, count)
- AI Shelf Scan (camera → ISBN counts)
- On-the-floor stock adjustment (quick add/remove with reason codes)
- Signature capture for deliveries

Built so a picker on a phone never has to walk back to a desk to log a count or confirm a pick.

Real-time across devices

When a count is recorded on a phone, the manager's dashboard updates within ~250 ms. This is not polling — it's a true WebSocket push from the database via Supabase Realtime. Multiple staff can pick from the same warehouse without stepping on each other; stock numbers stay coherent.

Email + notifications

Order events trigger transactional email:

- Order received → confirmation to requester

- Order approved → notification to requester
- Order shipped → tracking email
- Order signed → receipt to both signer and requester
- Weekly digest → summary of stock movements, top requests, overdue rentals

All emails are deliverable (SPF/DKIM/DMARC properly configured at the domain level) and bounce-tracked via Resend.

Permissions — five roles, three layers

Role	Day-job
Owner	Workspace founder. Billing + transfer.
Admin	Full operational control short of billing.
Manager	Approves orders, generates PDFs, runs reports, edits team. Most managers live here.
Staff	Picks, packs, receives, adjusts stock. Scoped to assigned warehouses.
Viewer	Read-only. Can submit order requests. Can be restricted to specific categories.

Permissions are enforced at three layers — UI (button hidden), application (server rejects the call), and database (row-level security policy returns zero rows). All three must agree. This is the difference between "we hide the button" and "the system literally cannot do that."

A viewer can be category-restricted — e.g., a parent volunteer who should only see Books, not Wellness Items. The admin checks the categories they're allowed to see; the volunteer never sees the rest, on any surface (lists, search, AI assistant, reports).

What makes it different from the alternatives

vs. Excel + email

Excel is what every warehouse starts with. The problem isn't that Excel can't represent inventory — it's that:

- It doesn't enforce who can edit what
- It doesn't have a real-time view across multiple users
- It doesn't keep a history of changes (you can see today's number but not the path that got you there)
- It doesn't scale past one warehouse without becoming a spreadsheet-of-spreadsheets nightmare
- It can't be queried by an AI assistant
- It can't generate a signed PDF receipt

StockPilot is what Excel becomes when you take it seriously.

vs. Generic WMS (NetSuite, Cin7, Fishbowl)

Big-name WMS systems are powerful but built for:

- Companies with full-time inventory analysts
- Multi-month implementation cycles
- \$30-150K/year licenses
- Customizations that require certified consultants

Most small/mid-size warehouses don't need 80% of what those systems do. The other 20% they DO need is what StockPilot focuses on: receive, slot, pick, ship, count, report.

The trade-off is that StockPilot is purpose-built and pragmatic, not endlessly configurable. If you need a workflow we don't have, we add it in a sprint, not a quarter.

vs. POS-add-on inventory (Microix on top of MIP Fund Accounting, Square inventory, etc.)

POS-side inventory products are built around the cash register. They track on-hand quantities well but stumble at:

- Pick lists in rack order
- Multi-warehouse scope enforcement
- Reservations vs. actual stock
- Cycle counts beyond simple "recount on demand"
- Public order links (no account needed)
- Rentals as a distinct lifecycle

StockPilot can sit alongside Microix or any POS — taking the floor operations, leaving the accounting close-out to the POS. We've sketched an integration path (read items + POs from Microix nightly; push counts and adjustments back) but it's optional.

vs. paper

Paper picks lose 5-15% efficiency to:

- Walking the floor in the wrong order
- Not knowing real-time stock counts
- Lost slips
- Manual reconciliation at end-of-day

StockPilot's pick slip is generated in rack order from the moment a manager approves the order. The picker walks one path. Confirmations happen on the phone in real time.

Security + accountability (audit-friendly)

Built for organizations that get audited:

- Every quantity change → **stock_movements** row with user + timestamp + reason + reference. Never deleted. Adjustments via offsetting rows.
- Every privileged action (role change, category access change, rental cancel, order deny) → **activity_logs** row. Visible in the audit log report. Manager+ only.
- Photo evidence: items have multiple photos; signatures stored as PNG data URIs on the order row; cycle counts can attach photos per line.

- All data lives in Postgres (Supabase). Backed up nightly. Point-in-time restore available.
- Item-level RLS on inventory: a deleted org's data is hidden, not lost. A revoked user immediately loses access.
- Public order link uses a per-org rotatable token. If the link leaks, rotate the token without affecting the workspace.
- Honeypot fields + per-IP/email/token rate limits on the public form keep bots out.

For end-of-period reconciliation:

- "What changed on item X between two dates?" → one query.
- "What did user Y do last quarter?" → one query.
- "Why is the on-hand quantity what it is?" → traceable through the immutable ledger.

This is the rigor a paper or Excel system can't deliver no matter how disciplined the operator is.

Hardware + IT requirements

What the warehouse needs:

What	Required?	Notes
Computer with a modern browser (Chrome, Safari, Firefox, Edge)	Yes	Office laptop is fine. No installed software.
Smartphones (iOS 15+ or Android 10+) for floor staff	Yes	Bring your own. The Expo app is on TestFlight / Google Play (private).
Barcode scanner	Optional	Bluetooth scanners work. Most phones have a built-in camera scanner that's faster than typing.
Wi-Fi reaching all racks	Recommended	If a section drops, the app caches and syncs when reconnected.
Label printer (Brother / Zebra)	Optional	Item labels print directly from /dashboard/inventory/labels .

What the IT team needs to do:

- Approve the cloud service (it's already deployed at [stockpilotusa.com](#))
- Add users / invite team members
- That's it. There is no on-prem server. No VPN. No database installation.

Roadmap (what's coming, prioritized by impact on warehouse operations)

Near-term:

- **Microix integration** (read POs + item master nightly; push cycle-count adjustments back) — requested by the partner org
- **Bulk return** for rentals (return 12 items at once at end of event)

- **Calendar / availability view** for rentals (book a canopy for next Saturday)
- **Rental slip PDF** (printed receipt of what's out)
- **Photo on rental return** (proof of condition)

Mid-term:

- **Cross-warehouse transfers with in-transit state** (DC4 → DC1 with proper "in transit" stock visibility)
- **Cycle count scheduling + reminders** (auto-create a session each quarter)
- **Supplier-grouped PO email** (one click sends each supplier their part of a multi-supplier PO)
- **AI Shelf Scan for non-books** (canopies, t-shirts via fine-tuned model)

Long-term:

- **Pickface optimization** (suggest re-slotting based on velocity)
- **Predictive reorder** (machine-learned reorder point per item per warehouse)
- **Customer portal for vendors** (let vendors confirm receipt of POs)
- **Compliance-mode audit pack** (one-click export of all audit-relevant data for a date range)

The roadmap is driven by what warehouse operators tell us hurts. Partnering brings that signal directly to the roadmap.

Why partner

If you're a warehousing expert reading this, here's the value of partnering — not the abstract, but the operational:

1. **Your domain knowledge plugs into a system that's actually moving.** Most "let's build a WMS" pitches are someone with a backend doing a 6-month dive into warehouse domain language. StockPilot is past that. The schema, the workflow, the audit trail are all built. Your input goes into product polish and roadmap, not foundation.
2. **Real production traffic to learn from.** L4L Fresno runs daily operations on it. Real picks, real cycle counts, real edge cases. You'll see how a real warehouse uses the system, which is the only training data that matters.
3. **A clean rolling product.** New features ship in days, not quarters. Average time from "we should have X" to "X is in production" has been 1-3 days for the last month of feature work. (Receipts: see the git log — about 40-50 commits per week of feature work + polish.)
4. **Built-in extensibility.** Adding a new module (e.g., yard management, returns processing, kit prep) follows a well-worn pattern: a service file, an action file, some UI pages. It's not a tear-up-the-foundation exercise — we know how to do it.
5. **A product that earns its keep on day one.** Unlike a 12-month WMS implementation, StockPilot is usable the day a new warehouse is added. The L4L pilot was operational in two weeks from "let's try it" to "we run all picking through it."
6. **A founder who codes plus a warehousing expert.** This combination is rare. Most warehouse software is built by software people guessing at the domain, OR by warehouse people who can't

ship. Partnering closes that gap permanently.

What you can do this week (concrete next steps)

If you're seriously evaluating:

1. **Walk through a typical day at your warehouse with the founder.** Bring real edge cases — "what happens when a vendor delivers 8 of an item that was POed for 10 and we accept it as a short receive?" Have them show the flow live in the system. Push on the edges.
 2. **Pick one bad workflow you live with today.** Picking in non-rack order. Cycle counts that take days. Lost signatures. Anything. Walk through how StockPilot handles that workflow and ask hard questions about edge cases.
 3. **Sit a junior staff member down with the mobile app for 15 minutes and time them.** Do they figure it out? Without a manual. Without help. That's the operational durability test.
 4. **Look at the reports.** Open Inventory Valuation, Reorder Forecast, ABC. Imagine a board meeting where you have to defend the inventory position with these. Are they enough? What's missing?
 5. **Talk to L4L Fresno's manager.** They've been running on the system since April. They'll tell you, unfiltered, what's good and what still needs work.
-

A note on what this system is NOT

To set the right expectations:

- **Not a financial accounting system.** We don't replace QuickBooks, Microix, or MIP. We feed them. Adjustments and receives can be exported as journal entries; we don't post to a general ledger directly.
- **Not a shipping carrier integration.** We don't print UPS labels or rate-shop FedEx. We track that an order shipped and capture a signature; carrier integration is roadmap.
- **Not a manufacturing system.** We don't track work-in-progress, BOM unrolls, or production scheduling. Bundles are a light alternative for kit assembly but it's not MRP.
- **Not a point-of-sale.** No cash drawer, no receipt printer, no transaction tax math. If you sell at a counter, plug your POS in alongside.
- **Not infinitely configurable.** We make opinionated choices. If the workflow needs to be radically different, we discuss whether to add a configuration or to extend the model. We don't ship a "build your own WMS" engine.

What we ARE is a focused, opinionated, audit-friendly system for the operational core of warehouse work — receive, slot, reserve, pick, pack, ship, count, report — built by people who care about the rigor of that work.

Closing

The pitch reduces to this:

Modern WMS-grade rigor at small-warehouse cost, built so a real warehouse expert can use it on day one and improve it on day two.

The math is friendly. The data is durable. The audit trail is real. The roadmap is responsive. And there's a founder you can call when something needs to change.

If that's a system you'd want next to you for the next five years of warehouse work, the next step is a conversation.

For more depth:

- Engineering-level walkthrough: [docs/SYSTEM-GUIDE.md](#) (printable, 25 pages, plain English but tech-leaning)
- Active implementation roadmap: [BLUEPRINT.md](#)
- L4L Fresno production: [stockpilotusa.com](#)